CS CODESAGE

An AI Software Engineer that proves its own fixes.

Understand a codebase. Investigate a real bug. Propose a verified patch. Run the tests. **Never claim success without execution.**

Productothon 01 — Challenge 01: AI Software Engineer

A chatbot gives you a guess. A bug needs proof.

NORMAL AI CHATBOT

"Your login issue might be caused by incorrect state management. Try updating your useState logic."

— Never reads your actual files

— Never runs a test

— No evidence it's even right

CODESAGE

Found the bug in **Login.jsx:22**. State updates, but navigation is never called after success.

— Read the real file, cited the real line

— Applied a real patch, ran the real tests

— 1 failed → 0 failed. Verified fixed.

The gap between "sounds right" and "is right" is exactly what CodeSage closes.

Understand. Investigate. Fix. Verify.

CodeSage is an autonomous engineering agent — not a code generator. It reads a real codebase, investigates a real bug report with tool-calling evidence, proposes a patch, and proves the fix by running your own tests.

Understand

Indexes the repo, detects the stack and test command, greps for candidates.

Investigate

A real tool-calling loop reads files and traces logic before it concludes anything.

Fix

Generates a real unified diff and applies it to an isolated copy of your repo.

Verify

Runs your real test suite before and after. The verdict is computed, not claimed.

The real engineering workflow

1. **Connect the repository**
   Local path, Git URL, or drag-and-drop folder upload.
2. **Understand the codebase**
   File scanning, dependency and stack detection, symbol indexing.
3. **Investigate with evidence**
   Search, read real files, trace functions — cite file:line before concluding.
4. **Propose and apply a patch**
   A real unified diff, applied only inside an isolated workspace copy.
5. **Verify and report**
   Run tests before/after, compare real exit codes, produce an engineering report.

Three decisions that separate this from "an LLM wired to a code editor"

Evidence-based debugging

Every root cause cites a real **file:line** the model actually read via a tool call — never an invented explanation.

Verification-first

Proposed → applied → tested → **verified**. A fix is only "fixed" when real exit codes flip from failing to passing.

Safe by construction

Every patch lands in an isolated filesystem copy with its own git repo. Your original project is never touched.

Honesty is a feature: an unverifiable result is reported as *inconclusive*, never dressed up as a success.

A real bug. A real fix. A real test flip.

"After a user logs in, the app shows they're authenticated, but it never navigates to /dashboard."

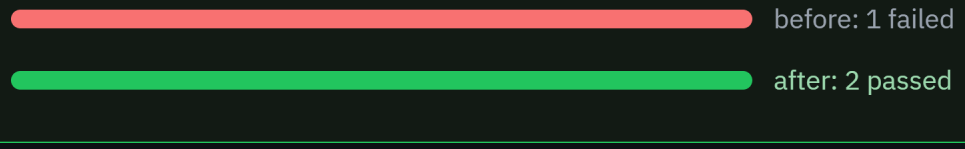
src/Login.jsx

```
  setLoggedIn(true);
-   if (loggedIn) {
+   if (ok) {
    navigate('/dashboard');
  }
```

Root cause

Checks the stale **loggedIn** state immediately after calling **setLoggedIn(true)** — React state updates aren't synchronous, so navigation never fires.

VERIFIED FIXED



Built on a real, running stack

Frontend
React + Vite + Tailwind
3-pane console, VS Code-style patched-file viewer

Backend
Express + TypeScript
Async jobs, disk-persisted state, durable fix storage

Reasoning
Anthropic · Groq · OpenRouter
Automatic fallback chain on rate limits

Patching
git apply + diff fallback
Auto-repairs malformed LLM diffs, one retry on failure

Verification
npm · pytest · node --test
Auto-detected, run before and after, in isolation

Tested
42 passing unit tests
Covering the pipeline, patching, and orchestration

Beyond one bug: a multi-bug orchestrator

Real BUG_REPORT.md files describe several unrelated issues at once. CodeSage splits them, fixes what it can, and never lets one failure take down the batch.

Structural splitting

Detects real "Bug N" headings for free — zero LLM calls — and rejects any split that looks like a generic procedure, not an actual bug.

Sequential, regression-tested

Each fix lands on a shared workspace; the full suite reruns after every patch to catch a new fix breaking an old one.

Failure-isolated

An inconclusive sub-bug rolls back and the batch continues — proven on a real 4-bug app: 3 fixed, 1 isolated, 4/4 passing after.

From a bug report to an evidence-backed fix — without the manual search.

4

Real test projects verified end-to-end — Python, Node, React, full-stack

3

LLM backends chained with automatic rate-limit fallback

0

Fabricated results — every verdict traces to a real exit code

For a developer, that's hours of manual tracing replaced by a verified diff and a test run. For a team, it's a debugging agent that never overstates what it actually did.

CS CODESAGE

Don't trust the fix. Verify it.

A live web app, a real CLI, and a genuine debugging agent — ready to demo end-to-end today.

Challenge 01 — AI Software Engineer

Productothon 01